Project 2 – Metaheuristics

Alejandro Fernández, Antoni Łopacz

Curso 2025

May 30, 2025

Contents

1 Problem 1: Fire Station Location Problem **3**
1.1 Local-search Method: Simulated Annealing 3
1.2 Population-based Method: Genetic Algorithm 4
1.3 Solution Visualization 5
1.4 Simulated Annealing Parameter Sweep Results 6
1.5 Genetic Algorithm Parameter Sweep Results 6

2 Problem 2: Bin Packing Problem **8**
2.1 Mathematical Formulation 8
2.2 Theoretical lower bound l 9
2.3 Applications of the Bin Packing Problem 9
2.4 Essential previus knowledge 10
2.5 Bin Packing Problem Variants and Problem Selection 11
2.6 1-d Offline Problem solving 13
2.6.1 Implementation of Simulated Annealing for the Bin Packing Problem 13
2.6.2 Implementation with Genetic Algorithm 16

Introduction

1 Problem 1: Fire Station Location Problem

The **Fire Station Placement Problem (FSPP)** is a combinatorial optimization problem defined on an undirected graph $G = (V, E)$, where

- $V = \{v_1, v_2, \ldots, v_n\}$ is the set of neighborhoods,
- $E \subseteq \{\{v_i, v_j\} \mid v_i, v_j \in V\}$ is the set of adjacency relations.

The goal is to select a minimum-cardinality subset $D \subseteq V$ of neighborhoods in which to build fire stations such that every neighborhood $v \in V$ either hosts a station or is adjacent to at least one station. Formally,

$$\forall\, v \in V: \quad v \in D \quad \text{or} \quad \exists\, u \in D \text{ with } \{u, v\} \in E.$$

This problem is equivalent to finding a minimum dominating set in G, and is known to be NP-hard in general. Practical applications of the FSPP include the strategic siting of emergency facilities, placement of service hubs in telecommunication networks, and allocation of resources in urban planning to ensure rapid response coverage with minimal infrastructure cost.
For the implementation, we can define binary decision variables

$$x_i = \begin{cases} 1, & \text{if } v_i \in D, \\ 0, & \text{otherwise,} \end{cases} \qquad i = 1, \ldots, n,$$

and formulate the optimization problem as

$$\begin{aligned} &\min_{x \in \{0,1\}^n} \sum_{i=1}^{n} x_i, \\ &\text{subject to: } x_i + \sum_{j:\, \{v_j, v_i\} \in E} x_j \;\geq 1, \quad \forall\, i = 1, \ldots, n. \end{aligned}$$

1.1 Local-search Method: Simulated Annealing

Simulated Annealing (SA) is a stochastic local-search heuristic inspired by the physical process of annealing in materials science. The algorithm begins at a high "temperature" T_{init}, which allows occasional uphill moves (i.e., acceptance of worse solutions) in order to escape local optima. As the search proceeds, the temperature is gradually reduced according to a cooling factor α, guiding the system toward convergence on low-cost (high-quality) solutions. At each iteration, a candidate solution y is generated from the current solution x, and is accepted with probability

$$P(\text{accept}) = \begin{cases} 1, & C(y) < C(x), \\ \exp\big((C(x) - C(y))/T\big), & \text{otherwise,} \end{cases}$$

where $C(\cdot)$ denotes the cost function and T the current temperature. This acceptance criterion balances exploration and exploitation, permitting the algorithm to both refine promising solutions and escape suboptimal regions of the search space.

SA Implementation Details

Initial solution: Two strategies:

- *Greedy*: iteratively add the vertex covering the largest number of still-uncovered neighborhoods until coverage is complete.
- *Random*: randomly shuffle all vertices and add them in order until the graph is dominated.

Cost function $C(D)$: For candidate set D:

$$C(D) \;=\; |D| \;+\; \text{penalty} \times (\text{number of uncovered nodes}),$$

where uncovered nodes are those neither in D nor adjacent to any vertex in D.

Neighborhood move : From current set D, choose at random among:

- *Add*: insert one random vertex not in D.
- *Remove*: delete one random vertex from D.
- *Swap*: remove one in-set vertex and add one outside.

Temperature schedule :

$$T \leftarrow \alpha\, T \quad \text{after each evaluation,}$$

starting from T_{init} down until T_{final}. An inner loop of up to $N_{\max}$ evaluations caps iterations per temperature.

1.2 Population-based Method: Genetic Algorithm

A Genetic Algorithm (GA) is a population-based metaheuristic inspired by natural evolution. We maintain a population of candidate solutions: binary chromosomes $x \in \{0,1\}^n$, where $x_i = 1$ indicates placing a station at vertex v_i. Each generation involves selecting parents according to their fitness (inverse cost), recombining them via crossover, applying random mutations, and optionally preserving a few elite solutions. Over successive generations, the population "evolves" toward improved dominating sets. The fitness objective is derived from the same penalized cost used in SA:

$$C(x) \;=\; \sum_{i=1}^{n} x_i \;+\; \lambda \cdot \big|\{v \notin D(x) \;\wedge\; \forall u \in D(x) : \{u, v\} \notin E\}\big|,$$

where λ is a large penalty for each uncovered neighborhood, and $D(x) = \{v_i : x_i = 1\}$.

GA Implementation Details

The initial population of size N_{pop} is generated by creating each chromosome with a random number of ones, uniformly between 5% and 30% of the n vertices, then placing that many ones at random positions. At each generation, we first optionally carry over the top N_{elite} chromosomes (elitism). To fill the remainder of the next population, we repeat until we have N_{pop} offspring:

- **Parent selection**: tournament selection of size k: choose k random individuals and select the one with lowest cost.

- **Crossover**: with probability p_{cross} we apply uniform crossover by swapping each gene independently with probability 0.5; otherwise, parents pass unchanged.
- **Mutation**: each bit of each offspring is flipped with independent probability p_{mut}.
- **Local search** (optional): with user flag, apply a 1-flip steepest-descent hill-climber to each mutated child, accepting any single-bit flip that strictly reduces $C(x)$.

After forming the new generation, we recompute costs $C(x)$, track the best individual of the generation, and update the overall best. This process repeats for G generations, yielding a high-quality approximate minimum dominating set.

1.3 Solution Visualization

To observe the behavior and progression of the implemented optimization algorithms, a dynamic visualization module was developed using Matplotlib. For each algorithm, the graph is rendered with nodes representing locations and edges indicating adjacency. Node positions are determined using a custom placement algorithm designed to minimize overlaps, or loaded from a pre-computed layout file for consistency across runs. During the optimization process, nodes selected as part of the current candidate dominating set (i.e., fire stations) are distinctly colored (e.g., blue). Nodes covered by this set are highlighted in a secondary color (e.g., light green), while any remaining uncovered nodes are marked in a contrasting color (e.g., red) to clearly indicate solution infeasibility.The visualization can either present two panes: one displaying the current or generation's best solution, and the other tracking the overall best solution found so far, or only present one, the overall best pane. The dual view allows for simultaneous observation of the algorithm's exploration and exploitation phases. The sequence of these rendered frames is compiled into a GIF animation, providing an intuitive understanding of the search trajectory and convergence characteristics, with relevant metrics such as temperature or generation number displayed alongside the solution cost.

Due to inherent PDF limitations, the visualizations are not included in the document. Instead, they are provided in a separate folder and presented within a Jupyter Notebook.

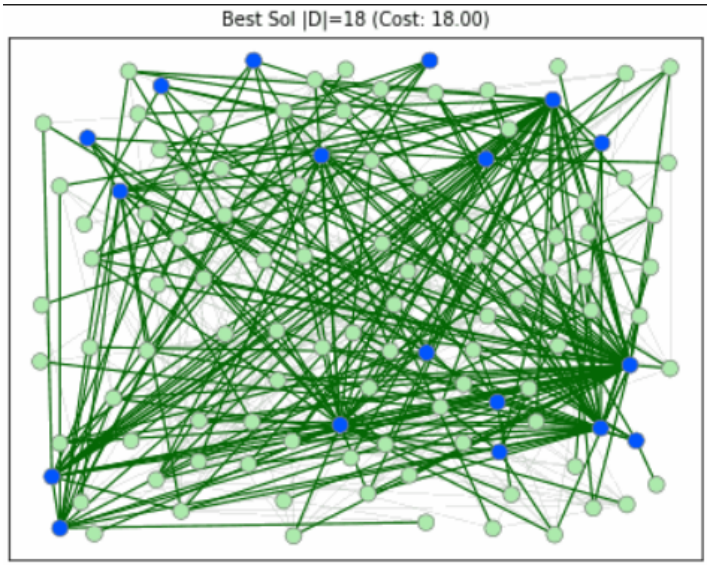


Figure 1: Hypothetical visualization of the best-found solution

1.4 Simulated Annealing Parameter Sweep Results

This section presents the results from a parameter sweep performed for the Simulated Annealing algorithm. The table below summarizes the best cost found for various combinations of initial temperature, final temperature, cooling rate (alpha), number of iterations, and initialization strategy.

Table 1: Simulated Annealing Parameter Sweep Results for MDS (Grid)

Initial Temp	Final Temp	Alpha	Iterations	Init Strategy	Best Cost
100.0	0.01	0.7	500	greedy	18.0
100.0	0.01	0.85	500	greedy	18.0
200.0	0.01	0.7	500	greedy	18.0
100.0	0.01	0.99	500	greedy	18.0
200.0	0.01	0.99	500	greedy	18.0
200.0	0.01	0.85	500	greedy	18.0
200.0	0.01	0.99	500	random	23.0
100.0	0.01	0.99	500	random	25.0
200.0	0.01	0.7	500	random	67.0
200.0	0.01	0.85	500	random	67.0
100.0	0.01	0.85	500	random	79.0
100.0	0.01	0.7	500	random	81.0

From Table 1, we can draw distinct conclusions based on the initialization strategy employed for the Simulated Annealing (SA) algorithm. When the SA search was initiated with a greedy heuristic, the resulting Best Cost consistently remained at 18.0 across all tested SA parameter configurations. This indicates that the greedy initialization itself produces a high-quality solution for this specific graph instance. The SA algorithm, when starting from this strong initial solution, was unable to discover any further improvements within the explored parameter space and iteration limits. Essentially, the greedy solution appears to be a potent local (and potentially global) optimum that the SA, under these conditions, did not escape or improve upon.

In contrast, when employing a random initialization strategy, the SA algorithm demonstrated a greater sensitivity to its operational parameters. The Best Cost varied significantly, ranging from 23.0 to 81.0. This variability highlights that the choice of SA parameters (Initial Temp, Alpha, etc.) plays a crucial role in guiding the search when starting from a less optimized, random point. For instance, configurations with a higher Alpha (e.g., 0.99), implying a slower cooling schedule, generally achieved better costs (e.g., 23.0 and 25.0) compared to those with a lower Alpha (e.g., 0.7, resulting in costs like 67.0 and 81.0) when Iterations were kept at 500. This suggests that a more gradual cooling allows the SA more opportunity to explore and escape poorer local optima when not starting with an already strong solution.

1.5 Genetic Algorithm Parameter Sweep Results

Top 10 Performing GA Configurations

Table 2 displays the ten parameter configurations that yielded the lowest (best) costs. It is noteworthy that all these top configurations achieved a ‘Best Cost‘ of 18.0. This suggests that for this problem instance, multiple GA parameter sets are capable of finding this high-quality solution, which matches the best solution found by the Simulated Annealing algorithm with greedy initialization. The common characteristic among these top runs is the utilization of local search.

In the implemented Genetic Algorithm, enabling this option means that every individual offspring, after undergoing crossover and bit-flip mutation, is then further refined by employing a simple but effective hill-climbing strategy: it explores the 1-flip neighborhood of the chromosome by testing all possible single bit-flips, and greedily adopts the flip that most reduces the solution cost, repeating this process until a local optimum (with respect to single bit-flips) is reached.

Table 2: Top 10 GA Configurations by Best Cost Found

Pop	Gens	Tourn	Cross	Mut	Elitism	Elite	LS	Best Cost
20	20	3	0.9	0.02	False	0	True	18.0
20	20	3	0.9	0.05	True	2	True	18.0
20	20	3	0.9	0.05	True	10	True	18.0
20	20	3	0.9	0.05	False	0	True	18.0
20	20	10	0.7	0.02	True	2	True	18.0
20	20	10	0.7	0.02	True	10	True	18.0
20	20	10	0.7	0.02	False	0	True	18.0
20	20	10	0.7	0.05	True	2	True	18.0
100	50	3	0.9	0.02	False	0	True	18.0
100	50	3	0.9	0.05	True	2	True	18.0

Bottom 5 Performing GA Configurations

To illustrate the range of outcomes, Table 3 shows the five parameter configurations that resulted in the highest (worst) costs. These configurations typically involve smaller population sizes or generation counts, lower crossover rates, or the absence of local search, leading to significantly poorer solutions, with one configuration resulting in a cost as high as 491.0. This underscores the importance of appropriate parameter settings and algorithmic components like local search for achieving good performance with GAs.

Table 3: Bottom 5 GA Configurations by Best Cost Found

Pop	Gens	Tourn	Cross	Mut	Elitism	Elite	LS	Best Cost
20	20	3	0.7	0.05	True	2	False	48.0
100	20	3	0.7	0.05	False	0	False	49.0
20	20	3	0.7	0.05	True	10	False	51.0
20	20	3	0.9	0.05	False	0	False	54.0
20	20	3	0.9	0.02	True	10	False	491.0

2 Problem 2: Bin Packing Problem

The **Bin Packing Problem (BPP)** is a classic combinatorial optimization problem where the goal is to pack a set of items of different sizes into the minimum number of containers (*bins*) with fixed capacity. This problem has numerous practical applications, ranging from logistics to storage and computational resource allocation.

This problem belongs to the class of **NP-hard** problems, which means that no algorithm is known to find optimal solutions in polynomial time for large instances. This computational complexity has motivated the development of numerous **heuristics and approximation methods** to obtain satisfactory solutions within reasonable time frames.

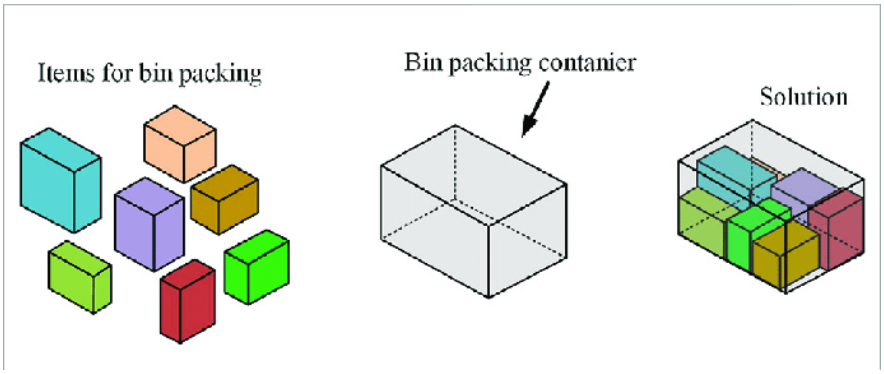


Figure 2: Example

2.1 Mathematical Formulation

Given a set of n items, each with a size $a_i \in R^+$, and containers of fixed capacity C, the objective is to minimize the total number of containers required to pack all items without exceeding the capacity of any container.

Decision Variables

- $x_{ij} = \begin{cases} 1, & \text{if item } i \text{ is placed in bin } j \\ 0, & \text{otherwise} \end{cases}$
- $y_j = \begin{cases} 1, & \text{if bin } j \text{ is used} \\ 0, & \text{otherwise} \end{cases}$

Mixed Integer Programming Model

$$\text{Minimize} \quad \sum_{j=1}^{n} y_j \tag{0}$$

$$\text{Subject to:} \quad \sum_{j=1}^{n} x_{ij} = 1 \quad \forall i = 1, \ldots, n \tag{1}$$

$$\sum_{i=1}^{n} a_i x_{ij} \leq C y_j \quad \forall j = 1, \ldots, n \tag{2}$$

$$x_{ij} \in \{0, 1\}, \quad y_j \in \{0, 1\} \tag{3}$$

Explanation of the constraints:

- **(0)** Objective function: Minimize the total number of bins used.
- **(1)** Each item must be placed in exactly one bin.
- **(2)** The total size of all items assigned to a bin cannot exceed the bin's capacity V — this constraint is only enforced if the bin is marked as used ($y_j = 1$).
- **(3)** Binary variables: $x_{ij} = 1$ if item i is assigned to bin j, and $y_j = 1$ if bin j is used.

2.2 Theoretical lower bound l

Before applying any optimization algorithm, it is useful to establish a theoretical lower bound on the number of bins required. This allows us to evaluate the quality of the solutions obtained by comparing them against this reference value. A commonly used bound is based on the total volume of items and the fixed bin capacity. Specifically, the minimum number of bins required, denoted as N, must satisfy the following inequality:

$$N \geq \left\lceil \frac{\sum_{i=1}^{n} S_i}{C} \right\rceil \tag{1}$$

where S_i represents the size of item i ($i = 1, \ldots, n$), and C is the fixed capacity of each bin. This expression assumes that bins are filled perfectly, without any wasted space, which is rarely achievable in practice. Nevertheless, it provides a useful benchmark to assess how close a given solution is to the best possible outcome.

2.3 Applications of the Bin Packing Problem

The bin packing problem, beyond its theoretical significance as a classic NP-hard optimization problem, has a wide range of practical applications across various industries and fields. Some of the most prominent applications include:

- **Logistics and Transportation:** Bin packing algorithms are extensively used for loading trucks, shipping containers, and pallets, where the goal is to maximize space utilization while respecting weight and volume constraints. This is crucial for reducing transportation costs and improving efficiency.

- **Data Storage and File Backup:** In information technology, bin packing is applied to problems such as creating file backups on storage media (e.g., CDs, DVDs, or hard drives), where files of varying sizes must be grouped to minimize the number of storage units used.
- **Network Design and Subnetting:** The problem appears in splitting a network prefix into multiple subnets of different sizes, where each subnet must fit within a given address space, analogous to packing items into bins.
- **Manufacturing and Cutting Stock:** In manufacturing, especially in industries dealing with materials like wood, metal, or textiles, bin packing models are used for cutting raw materials into pieces of specified sizes with minimal waste. This includes both one-dimensional (length) and multi-dimensional (area or volume) variants.
- **Cloud Computing and Resource Allocation:** Modern applications include allocating virtual machines or tasks to physical servers in data centers. Here, the objective is to minimize the number of servers (bins) used while ensuring that resource constraints (CPU, memory, etc.) are not violated.
- **VLSI Design and FPGA Technology Mapping:** Very Large Scale Integration (VLSI) is the process of creating integrated circuits by packing millions of transistors onto a single chip. Field-Programmable Gate Arrays (FPGAs) are special chips that can be reprogrammed to perform different digital functions. Bin packing is used to efficiently assign logic functions to the limited hardware blocks inside an FPGA, similar to packing items into bins.

These applications demonstrate the versatility and importance of the bin packing problem in both traditional and modern contexts. The ability to efficiently pack items into limited resources directly translates into cost savings, improved performance, and optimized operations in real-world systems.

2.4 Essential previus knowledge

First Fit Heuristic

The **First Fit** heuristic is one of the most classic and intuitive strategies for solving the Bin Packing Problem. Its philosophy is simple: place each item in the first bin that can accommodate it. If no current bin has enough remaining capacity, a new bin is opened.

Procedure. Given a list of items and a fixed bin capacity:

1. Iterate through the list of items in a given order.
2. For each item, scan the existing bins in the order they were created.
3. Place the item in the first bin where it fits without exceeding the capacity.
4. If no suitable bin is found, create a new bin and place the item there.

This heuristic is efficient in practice, with linear or near-linear time complexity. While it does not always yield an optimal solution, it often produces a reasonably good packing with minimal computational cost.

Usage in Our Implementation. In our project, a randomized version of the First Fit heuristic is used to construct feasible individuals for the initial population of the Genetic Algorithm. The same idea is implicitly applied in the construction of initial solutions for Simulated Annealing, ensuring that each generated solution satisfies the capacity constraint from the beginning.

Feasibility Strategies in Optimization

When solving constrained optimization problems, it is essential to ensure that candidate solutions satisfy the required constraints. Two common strategies to handle feasibility are described below:

Feasibility-by-construction. In this approach, the algorithm is designed to generate only feasible solutions. Each step of the solution construction ensures that no constraint is violated. This may involve checking constraint satisfaction during item placement or explicitly building the solution within the feasible space. As a result, infeasible solutions are never produced.

Feasibility-by-evaluation. This strategy allows the generation of infeasible solutions, which are then evaluated using a cost function that penalizes constraint violations. The penalty can be designed in various ways, often assigning an infinite or very high cost to infeasible candidates. This encourages the optimization process to favor feasible solutions over time, while still allowing a broader exploration of the search space.

Both strategies offer different advantages: feasibility-by-construction guarantees constraint satisfaction but can limit exploration, whereas feasibility-by-evaluation allows more flexibility but requires careful penalty design.

2.5 Bin Packing Problem Variants and Problem Selection

The Bin Packing Problem (BPP) encompasses a wide range of optimization problems that share a common structure: a set of items with defined sizes must be packed into containers (bins) while minimizing a cost function, typically the number of bins used. However, depending on the context and constraints, several distinct families of the BPP have emerged in the literature. These can be broadly categorized into two main groups: **offline** and **online** bin packing.

Offline Bin Packing Problems

In offline variants, the complete list of items to be packed is known in advance. This setting allows for pre-processing, sorting, and optimization strategies that consider global information about the input. It is particularly well-suited for applying metaheuristics and exact methods.

The main offline subtypes include:

- **1D Bin Packing (Classical):** Each item has a single size attribute (e.g., volume or weight). All bins have identical capacity. This is the most widely studied version of the problem.
- **Multidimensional Bin Packing:** Items have multiple attributes (e.g., weight and volume), and bins have a vector of capacities. An item can only be assigned to a bin if all constraints are satisfied across dimensions.
- **Variable-Sized Bin Packing:** There are multiple types of bins available, each with different capacities and possibly associated costs. The objective may include minimizing total cost instead of just the number of bins.

- **Bin Packing with Cardinality Constraints:** Each bin may hold only a limited number of items, regardless of their sizes.
- **Bin Packing with Conflicts or Dependencies:** Some items cannot be packed together due to incompatibilities or precedence constraints.

Online Bin Packing Problems

In contrast, online bin packing assumes that items arrive sequentially, one at a time, and decisions must be made immediately without knowledge of future items. This scenario models real-time systems and streaming applications but imposes stricter algorithmic constraints.

Common online subtypes include:

- **Online 1D Bin Packing:** Items arrive one by one, and the algorithm must assign each item to a bin as it arrives. No item reordering is allowed.
- **Bounded Space Online Packing:** The number of open bins is limited at any time, introducing additional memory constraints.
- **Online with Advice:** Algorithms receive partial information or predictions about future items, improving performance at the cost of relying on assumptions.

Problem Selection and Justification

Given the wide range of variants, this project focuses on the **offline one-dimensional bin packing problem with homogeneous bins**. This version is the most thoroughly studied in the literature and serves as the standard benchmark for evaluating heuristic and metaheuristic algorithms.

Moreover, the offline setting is essential for applying the methods explored in class—such as *Simulated Annealing*, *Tabu Search*, *Best Improvement*, and population-based techniques like *Genetic Algorithms*—which rely on iterative exploration of the solution space. These methods require full knowledge of the item list to generate, evaluate, and improve candidate solutions.

By selecting the classical offline 1D BPP, we ensure a well-defined experimental environment, practical relevance, and alignment with both the theoretical foundations and algorithmic techniques studied in the course.

2.6 1-d Offline Problem solving

Following the design outlined in Table 4, we implemented a tailored version of the Simulated Annealing (SA) metaheuristic to address the classical offline one-dimensional Bin Packing Problem. And in Table 5 we implemented as well the genetic algorithmm to this problem

2.6.1 Implementation of Simulated Annealing for the Bin Packing Problem

Component	**Implementation in Bin Packing Problem (BPP)**
Input Data	A list of item sizes and a fixed bin capacity.
Solution Representation	A list of bins, where each bin is a list containing items assigned to it.
Initial Solution	Constructed using a simple greedy **First Fit heuristic**: items are added to the first bin that has enough capacity, or a new bin is created. This does no guarantee an optimal but is a good starting point
Cost Function	The number of bins used (i.e., the length of the list of bins).
Neighborhood Generation	A neighbor is created by moving an item from one bin to another, provided the capacity constraint is not violated. If a bin becomes empty, it is removed.
Acceptance Criterion	Standard Metropolis criterion: always accept better solutions; accept worse solutions with a probability that decreases with temperature.
Cooling Schedule	Geometric cooling: temperature is multiplied by a constant factor after each iteration.

Table 4: Simulated Annealing components adapted for the Bin Packing Problem

Feasibility Constraint Handling

A critical aspect of the Bin Packing Problem is ensuring that the total size of items assigned to any bin does not exceed its fixed capacity V. In our implementation of Simulated Annealing, this feasibility constraint is handled using a **feasibility-by-construction** strategy.

During both the generation of the initial solution and the creation of neighbor solutions, items are only placed into bins if the resulting total size remains within the capacity limit. This ensures that all candidate solutions explored by the algorithm are valid, eliminating the need for penalty functions or repair mechanisms.

Although alternative strategies exist—such as allowing temporary violations with penalty terms in the objective function, or repairing infeasible solutions after generation—we opted for this direct approach to maintain simplicity and computational efficiency.

Problem Statement

The problem addressed in this section corresponds to the classical one-dimensional offline Bin Packing Problem. A set of items, each characterized by a single numerical attribute (interpreted here

as weight), must be assigned to a set of identical bins with fixed capacity. The goal is to pack all items using the minimum number of bins, without exceeding the bin capacity constraint.

For this experiment, we generated synthetic datasets of varying sizes (50 to 200 items), with item weights randomly sampled between 1 and 10 units. Each bin has a fixed capacity of 15 units. This setting allows us to evaluate the behavior of metaheuristic algorithms in a controlled yet realistic environment.

Our synthetic data set was initially of 100 items that were generated with this weight

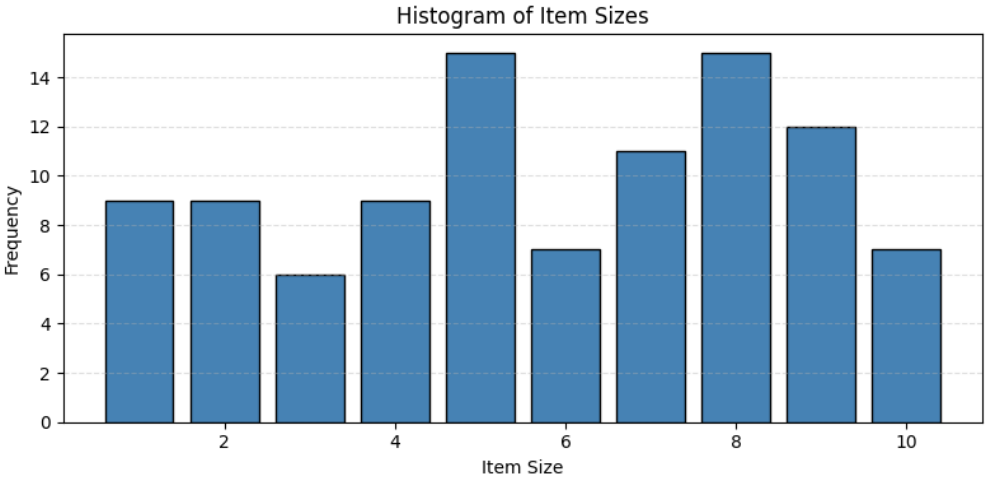


Figure 3: Weights distribution

We obtain that the optimal number of bins used is : **40**
with this representation :

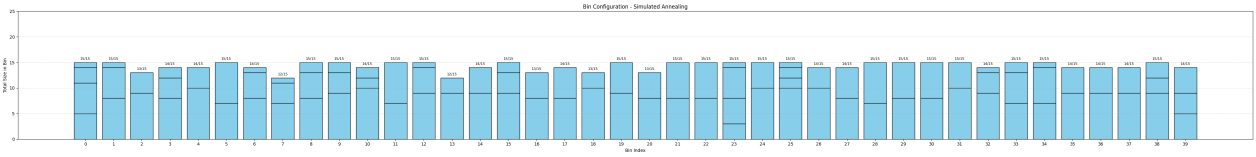


Figure 4: Bins distribution

Experiment: Effect of Dataset Size

To better understand the scalability of the Simulated Annealing algorithm, we conducted a set of experiments varying the number of items in the synthetic dataset. Specifically, we tested input sizes of 50, 100, 150, and 200 items. Each item weight was randomly sampled from a uniform distribution in the range [1, 10], and the bin capacity was fixed at 15 units and we tried also with 10 .

For each dataset size, we recorded the number of bins required to pack all items using Simulated Annealing. The results were plotted and analyzed to determine whether a linear relationship exists between the number of items and the number of bins used.

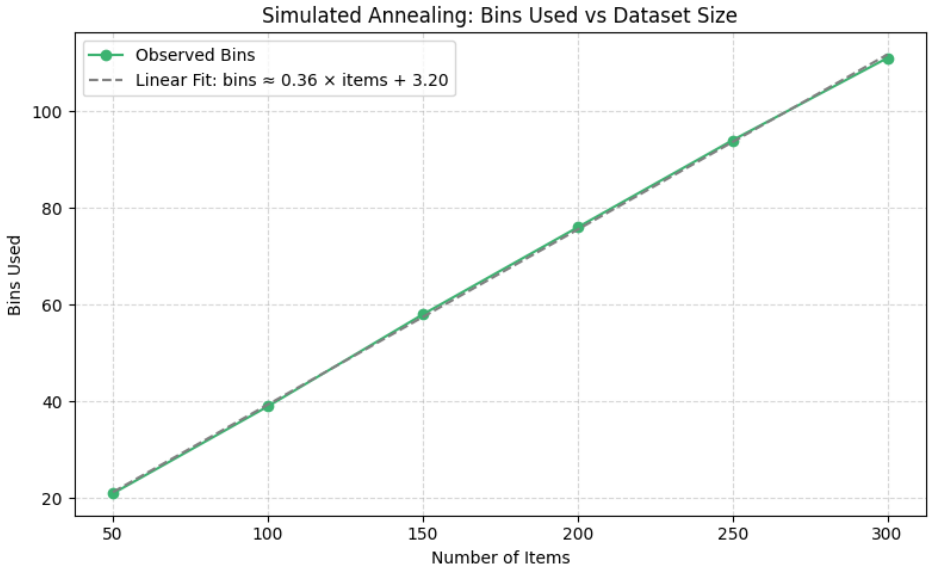


Figure 5: Bins used by Simulated Annealing vs. number of items (max item size = 15)

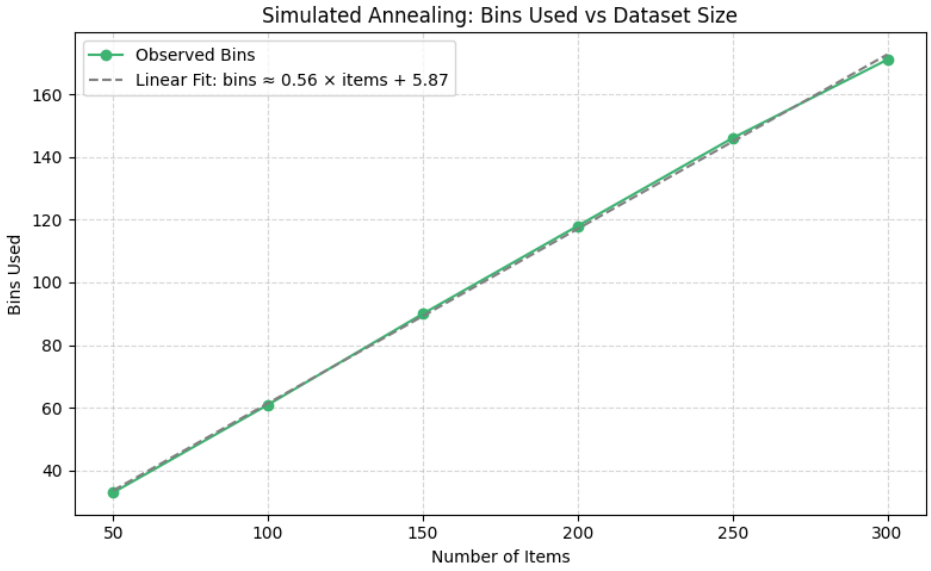


Figure 6: Bins used by Simulated Annealing vs. number of items (max item size = 10)

The graph above shows a near-linear trend, suggesting that the algorithm scales predictably with the problem size under these conditions.

Empirical Relationship Between Number of Items and Bins Required

As part of the analysis, we examined how the number of bins grows with respect to the number of items to be packed. To do this, we generated synthetic datasets of increasing size (e.g., 50, 100, 150, 200 items) and recorded the number of bins required in each case.

The experiments were conducted under two different bin capacity constraints: 15 units and 10

units per bin. The results showed a clear linear trend in both scenarios. By fitting a linear regression model, we obtained the following empirical relationships:

- For bin capacity of 15 units:
$$\text{Bins} = 0.36 \cdot \text{Items} + 3.2$$
- For bin capacity of 10 units:
$$\text{Bins} = 0.56 \cdot \text{Items} + 5.87$$

These equations reflect the intuitive result that tighter capacity constraints (i.e., smaller bin sizes) lead to a steeper growth in the number of bins needed as the number of items increases.

2.6.2 Implementation with Genetic Algorithm

In the context of the Genetic Algorithm applied to the Bin Packing Problem, each individual in the population is represented as a list of integers of length n, where n is the number of items to be packed. The i-th element of this list indicates the bin to which item i is assigned.

`[0, 2, 1, 0, 2, 3, 1]` $\Rightarrow$ item 0 goes to bin 0, item 1 to bin 2, etc.

This representation is compact and well-suited for standard genetic operations such as uniform crossover and mutation. However, it does not inherently enforce the feasibility constraint of the problem.

Feasibility Constraint. The main constraint of the Bin Packing Problem states that the total size of items assigned to any bin must not exceed its fixed capacity V:

$$\sum_{i \in \text{bin } j} s_i \leq V \quad \forall j$$

To ensure that the search space remains feasible from the outset, we apply a **feasibility-by-construction** strategy during the generation of the initial population. This is implemented through a randomized First Fit heuristic: each item is assigned to an existing bin that has sufficient remaining capacity, selected in random order; if no bin can accommodate the item, a new bin is created.

As for crossover and mutation operations, they can still lead to infeasible individuals. For this reason, the algorithm retains a **feasibility-by-evaluation** mechanism: each candidate solution is evaluated, and any that violate the bin capacity constraint are assigned an infinite cost and excluded from being selected in future generations.

This dual strategy ensures that the initial population is entirely feasible while allowing genetic diversity during evolution. Infeasible individuals are naturally discarded during the selection process, maintaining the overall integrity of the population.

Ensuring Feasibility in the Initial Population

In early versions of the algorithm, the initial population was generated by randomly assigning items to arbitrary bins. However, this often resulted in infeasible individuals in which the total size of items in some bins exceeded the fixed capacity. As a result, the entire population was sometimes discarded at the very beginning of the search due to the absence of feasible solutions.

To address this issue, we implemented a feasibility-by-construction strategy for the initial population. Specifically, we adapted a randomized version of the First Fit heuristic: for each item, we attempt to place it in an existing bin selected in random order; if it does not fit in any, a new bin is opened. This process ensures that each individual generated at initialization respects the bin capacity constraint:

$$\sum_{i \in \text{bin } j} s_i \leq V \quad \forall j$$

With this adjustment, all individuals in the initial population are guaranteed to be feasible, eliminating the need for additional filtering or penalization mechanisms at this stage of the algorithm.

Implementation for Bin packing

Component	Implementation in Bin Packing Problem (BPP)
Input Data	A list of item sizes and a fixed bin capacity.
Solution Representation	A list where the i-th element represents the bin assigned to item i.
Initial Population	Each individual is generated by randomly assigning each item to a bin index from 0 to $n-1$. We implement here the **Feasibility-by-construction** by defining a randomized version of the First Fit heuristic ensures that no item is assigned to a bin that would exceed its capacity.
Cost Function	The number of unique bins used, penalizing any solution where a bin exceeds its capacity. We use a **Feasibility-by-evaluation** at the time that if some bin is bigger than its capacity limit then we penalize it by adding and infinite value to the cost function.
Selection	Tournament selection: the best individual among a random subset of the population is chosen.
Crossover	Uniform crossover: each gene (item assignment) is inherited randomly from one of the two parents.
Mutation	A single item's bin assignment is changed to a new random bin.
Feasibility Handling	Feasibility-by-evaluation: solutions violating bin capacity are penalized with infinite cost. Also the implementation of Feasibility-by-construction in order to have a nice and completely functional initial population
Elitism	If enabled, the best individuals are directly carried over to the next generation.

Table 5: Genetic Algorithm components adapted for the Bin Packing Problem

It should be noted that during the evolutionary process, particularly after applying mutation or crossover, some individuals may become infeasible by exceeding the bin capacity constraint. To address this, a **feasibility-by-evaluation** approach is implemented in the cost function: any individual violating the constraint is penalized with an infinite cost, ensuring that such solutions are effectively excluded from selection in future generations.

Problem Statement

This section addresses the same one-dimensional offline Bin Packing Problem previously introduced. The task consists of assigning a set of items—each with an associated weight—to a minimal number of fixed-capacity bins, without exceeding the capacity in any bin.

To maintain consistency with the Simulated Annealing experiment, the synthetic dataset used here contains 100 items with weights uniformly sampled between 1 and 10. The bin capacity is set to 15 units. This setup allows for a direct comparison between the performance of different metaheuristic approaches under identical conditions.

We have our weights how they are distributed (We use the same as before of SA) :

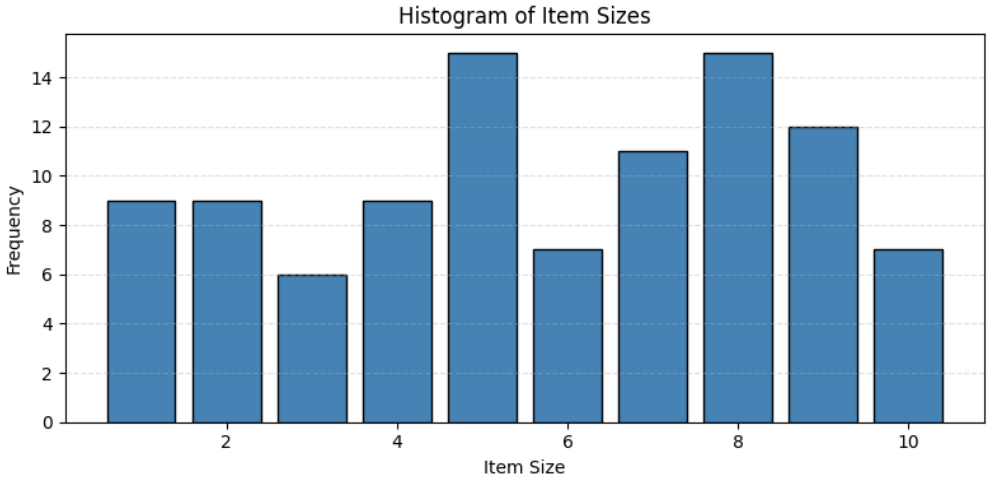


Figure 7: Bins used by Genetic algoritm vs. number of items (max item size = 15)

And bins : **40**

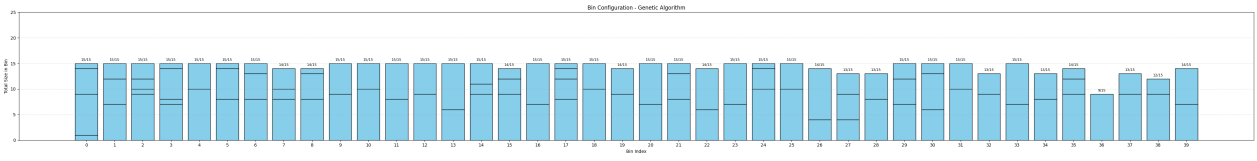


Figure 8: Bins used by Genetic algorithm vs. number of items (max item size = 10)

To explore whether there exists a correlation between the number of items and the number of bins required, we conducted experiments with datasets of increasing size. Letting us know the following correlation, also adding one modification by reducing the size od each bin to 10

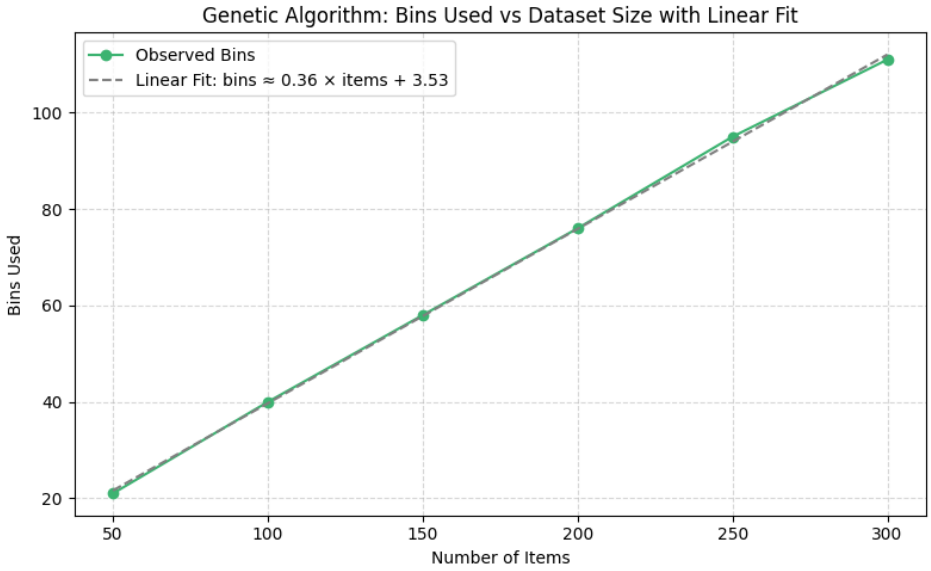


Figure 9: Bins used by Simulated Annealing vs. number of items (max item size = 15)

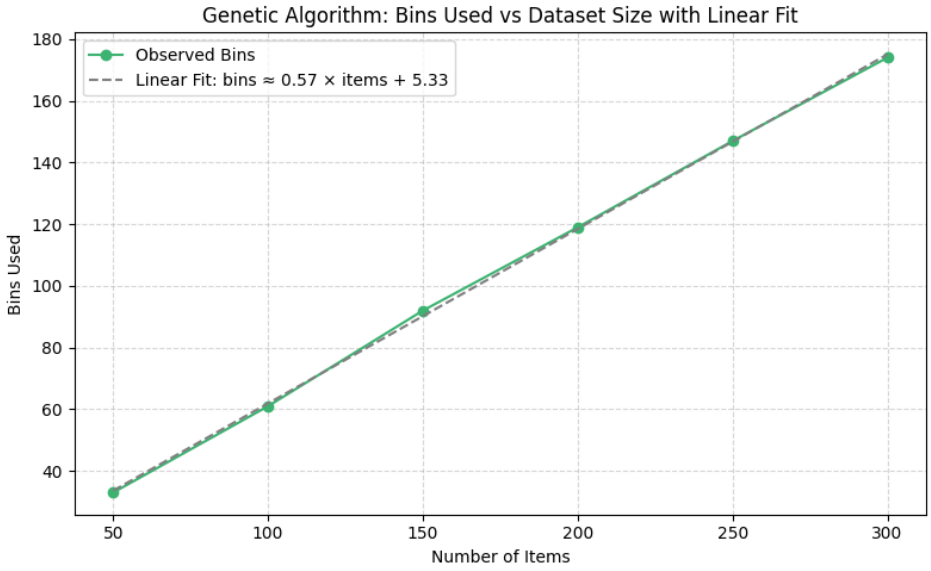


Figure 10: Bins used by Simulated Annealing vs. number of items (max item size = 10)

Linear Fit for Genetic Algorithm Results

For the Genetic Algorithm, we conducted the scaling experiment using two different bin capacities: 15 and 10 units. Initially, we focused only on the 15-unit case to explore the algorithm's performance under standard conditions. However, to enable a more complete comparison with Simulated Annealing and observe how the algorithm scales under tighter constraints, we extended the experiment to include the 10-unit scenario as well.

In both cases, we observed a linear relationship between the number of items and the number of bins required. The fitted regression models for each setting were as follows:

- For bin capacity of 15 units:

$$\text{Bins} = 0.36 \cdot \text{Items} + 3.56$$

- For bin capacity of 10 units:

$$\text{Bins} = 0.57 \cdot \text{Items} + 5.33$$

These equations show how the number of bins increases more rapidly as the bin capacity decreases, which aligns with the expected behavior of the bin packing problem.

Comparative Analysis: Simulated Annealing vs. Genetic Algorithm

Initial Observations and Linear Model Comparison

Both metaheuristics—Simulated Annealing and Genetic Algorithm—produced very similar results when applied to the 1D offline Bin Packing Problem under the same conditions. In most test cases, both algorithms converged to the same number of bins, especially when the dataset was relatively balanced and the problem was not overly constrained.
To quantify how each algorithm scales with increasing problem size, we fitted linear regression models to the number of bins required as a function of the number of items. The resulting equations were:

- **Simulated Annealing**
  - Capacity 15: $\text{Bins} = 0.36 \cdot \text{Items} + 3.2$
  - Capacity 10: $\text{Bins} = 0.56 \cdot \text{Items} + 5.87$
- **Genetic Algorithm**
  - Capacity 15: $\text{Bins} = 0.36 \cdot \text{Items} + 3.56$
  - Capacity 10: $\text{Bins} = 0.57 \cdot \text{Items} + 5.33$

The similarity in slopes between both algorithms under identical capacity settings indicates that they scale in a nearly equivalent way with respect to input size. However, the slightly lower intercept and slope observed for Simulated Annealing at capacity 10 suggest that it may be marginally more efficient when dealing with tighter packing constraints.

These linear models provide an initial quantitative basis for comparison and highlight that, despite following different search paradigms, both algorithms are capable of producing consistent and comparable solutions in terms of bin usage.

Convergence Plot Analysis: Identical Bin Count Across Iterations

To further analyze how each algorithm evolves during the search process, we plotted the number of bins used by the best solution at each iteration (for Simulated Annealing) or generation (for the Genetic Algorithm).

As shown in the figure above, both algorithms consistently maintained a bin count of 40 across all iterations. This flat line indicates that neither algorithm improved upon the initial solution during the search.

There are several key takeaways from this behavior:

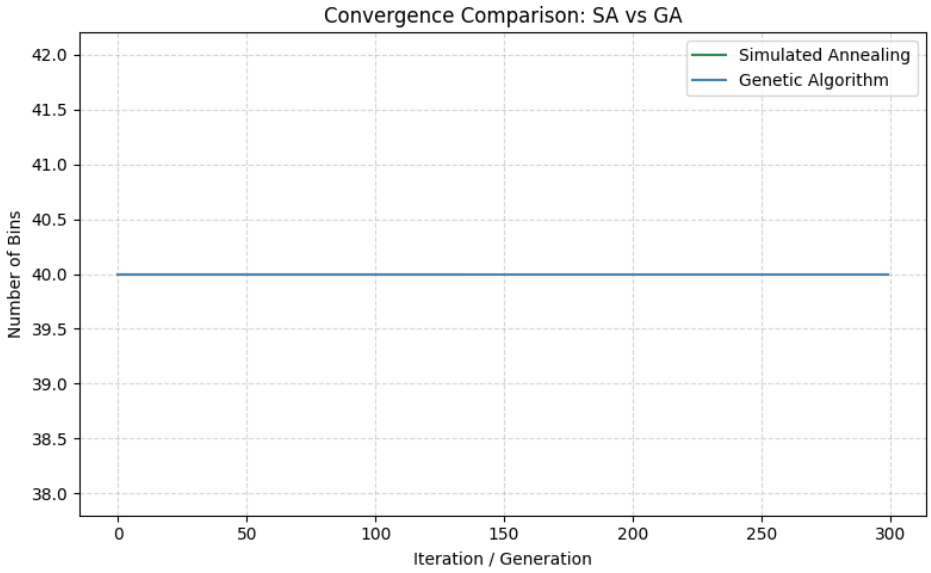


Figure 11: Convergence of Simulated Annealing and Genetic Algorithm (easy dataset)

- **Problem Simplicity:** The dataset appears to be relatively easy to solve. With item sizes ranging between 1 and 10 and a bin capacity of 15, many feasible packings exist. As a result, both algorithms are able to reach a good solution early in the process.

- **Offline Problem Characteristics:** Since this is an *offline* Bin Packing Problem, all item information is available from the start. This allows for stronger initial solutions, particularly when using heuristics like randomized First Fit. In contrast to online variants, where decisions are made incrementally and without full knowledge, offline problems tend to be more forgiving and easier to solve with basic strategies.

- **Simulated Annealing Stability:** The fact that Simulated Annealing remains stable at 40 bins suggests that the initial solution was close to optimal, and that the algorithm either could not escape this region or found no better solution in the landscape.

- **Genetic Algorithm Robustness:** The Genetic Algorithm also maintained 40 bins throughout all generations, highlighting its consistency—but also suggesting possible premature convergence to a local optimum, particularly due to limited diversity or a less disruptive mutation operator.

- **Low Discriminative Power in Easy Instances:** Since both methods converged to the same solution from the start, the experiment revealed little about their comparative strength. This led us to increase the difficulty of the dataset in subsequent experiments to obtain more informative results.

This experiment demonstrates the importance of instance difficulty in metaheuristic evaluation: when the problem is too simple, both methods may perform equally well, and differences in exploration capability may remain hidden.

Convergence with Increased Problem Difficulty

To reveal deeper behavioral differences between the algorithms, we increased the complexity of the dataset by narrowing the margin for packing. Specifically, item sizes were constrained to be larger (e.g., between 8 and 15 units) while maintaining the same bin capacity, thereby creating more tightly packed instances with fewer feasible configurations.

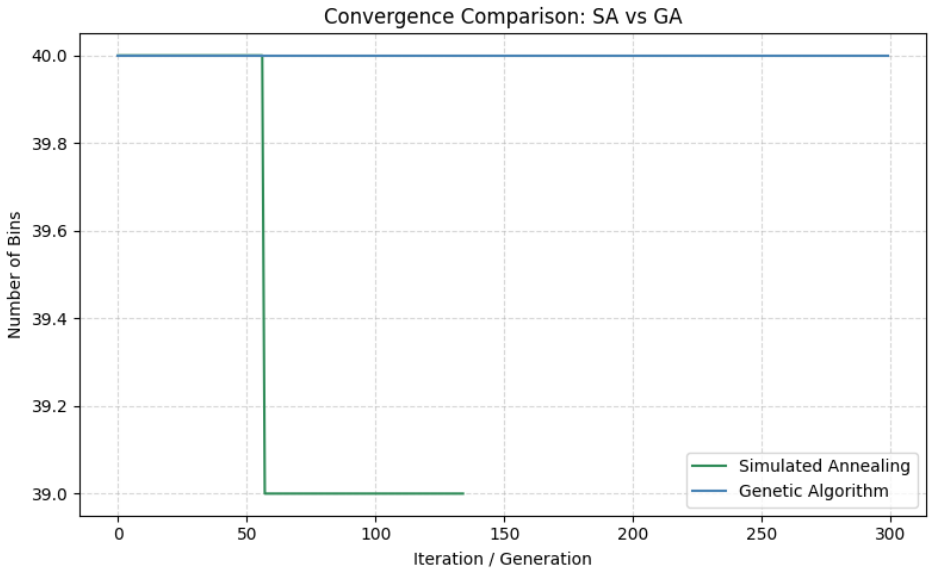


Figure 12: Convergence of Simulated Annealing and Genetic Algorithm (difficult dataset)

As shown in the figure, both algorithms initially start with a solution using 40 bins. However, around iteration 55, **Simulated Annealing finds a better configuration** that uses only 39 bins and remains stable at that value until the end. This improvement highlights its capacity to escape local optima by accepting worse solutions during early stages of the search.

In contrast, the **Genetic Algorithm remains flat at 40 bins**, indicating that it was not able to find a better configuration throughout the search. This stagnation may be due to limited exploration, lack of disruptive mutation, or premature convergence caused by insufficient population diversity.

This comparison confirms that increasing the problem's difficulty allows more expressive differences to emerge between metaheuristics. It reinforces the idea that Simulated Annealing's probabilistic exploration makes it more adaptable in constrained settings, while the Genetic Algorithm may require tuning or hybridization to improve performance in harder instances.

References

[1] Wikipedia, *Bin packing problem*, Disponible en: https://en.wikipedia.org/wiki/Bin_packing_problem, Último acceso: 20 de mayo de 2025.

[2] J. T. Peña et al., *A Binary Differential Evolution Algorithm for the Bin Packing Problem*, IEEE, 2020. Disponible en: https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9300141, Último acceso: 20 de mayo de 2025.

[3] StudySmarter, *Bin Packing Algorithms - Explanation*, Disponible en: https://www.studysmarter.co.uk/explanations/math/decision-maths/bin-packing-algorithms/, Último acceso: 20 de mayo de 2025.

[4] GeeksforGeeks. *Bin Packing Problem (Minimize number of used Bins)*. Available online at: https://www.geeksforgeeks.org/bin-packing-problem-minimize-number-of-used-bins/. Accessed on May 22, 2025.